

PL/SQL

Exercise 1 - Scenario 1

```
DECLARE
BEGIN
  FOR c IN (SELECT CustomerID,DOB FROM Customers) LOOP
    IF TRUNC(MONTHS_BETWEEN(SYSDATE,c.DOB)/12)>60 THEN
      UPDATE Loans
      SET InterestRate=InterestRate-1
      WHERE CustomerID=c.CustomerID;
    END IF;
  END LOOP;
END;
/
```

Exercise 1 - Scenario 2

```
ALTER TABLE Customers ADD IsVIP CHAR(1);
BEGIN
  FOR c IN (SELECT CustomerID,Balance FROM Customers) LOOP
    IF c.Balance>10000 THEN
      UPDATE Customers SET IsVIP='Y' WHERE CustomerID=c.CustomerID;
    END IF;
  END LOOP;
END;
/
```

Exercise 1 - Scenario 3

```
BEGIN
  FOR l IN (SELECT c.Name,l.EndDate FROM Loans l JOIN Customers c
  ON l.CustomerID=c.CustomerID
  WHERE l.EndDate BETWEEN SYSDATE AND SYSDATE+30) LOOP
    DBMS_OUTPUT.PUT_LINE('Reminder sent to ' || l.Name);
  END LOOP;
END;
/
```

Exercise 2 - Scenario 1

```
CREATE OR REPLACE PROCEDURE SafeTransferFunds(p_from NUMBER,p_to NUMBER,p_amt
NUMBER) IS
  bal NUMBER;
```

```

BEGIN
SELECT Balance INTO bal FROM Accounts WHERE AccountID=p_from FOR UPDATE;
IF bal<p_amt THEN
  RAISE_APPLICATION_ERROR(-20001,'Insufficient Funds');
END IF;
UPDATE Accounts SET Balance=Balance-p_amt WHERE AccountID=p_from;
UPDATE Accounts SET Balance=Balance+p_amt WHERE AccountID=p_to;
COMMIT;
EXCEPTION
WHEN OTHERS THEN
  ROLLBACK;
  DBMS_OUTPUT.PUT_LINE(SQLERRM);
END;
/

```

Exercise 2 - Scenario 2

```

CREATE OR REPLACE PROCEDURE UpdateSalary(p_id NUMBER,p_per NUMBER) IS
BEGIN
  UPDATE Employees SET Salary=Salary+(Salary*p_per/100)
  WHERE EmployeeID=p_id;
  IF SQL%ROWCOUNT=0 THEN
    RAISE NO_DATA_FOUND;
  END IF;
EXCEPTION
  WHEN NO_DATA_FOUND THEN
    DBMS_OUTPUT.PUT_LINE('Employee not found');
END;
/

```

Exercise 2 - Scenario 3

```

CREATE OR REPLACE PROCEDURE AddNewCustomer(
  p_id NUMBER,p_name VARCHAR2,p_dob DATE,p_bal NUMBER) IS
BEGIN
  INSERT INTO Customers VALUES(p_id,p_name,p_dob,p_bal,SYSDATE);
EXCEPTION
  WHEN DUP_VAL_ON_INDEX THEN
    DBMS_OUTPUT.PUT_LINE('Customer already exists');
END;
/

```

Exercise 3 - Scenario 1

```
CREATE OR REPLACE PROCEDURE ProcessMonthlyInterest IS
BEGIN
    UPDATE Accounts
    SET Balance=Balance*1.01
    WHERE AccountType='Savings';
END;
/
```

Exercise 3 - Scenario 2

```
CREATE OR REPLACE PROCEDURE UpdateEmployeeBonus(
p_dept VARCHAR2,p_bonus NUMBER) IS
BEGIN
    UPDATE Employees
    SET Salary=Salary+(Salary*p_bonus/100)
    WHERE Department=p_dept;
END;
/
```

Exercise 3 - Scenario 3

```
CREATE OR REPLACE PROCEDURE TransferFunds(
p_from NUMBER,p_to NUMBER,p_amt NUMBER) IS
    bal NUMBER;
BEGIN
    SELECT Balance INTO bal FROM Accounts WHERE AccountID=p_from;
    IF bal>=p_amt THEN
        UPDATE Accounts SET Balance=Balance-p_amt WHERE AccountID=p_from;
        UPDATE Accounts SET Balance=Balance+p_amt WHERE AccountID=p_to;
    END IF;
END;
/
```

Exercise 4 - Scenario 1

```
CREATE OR REPLACE FUNCTION CalculateAge(d DATE)
RETURN NUMBER IS
BEGIN
    RETURN TRUNC(MONTHS_BETWEEN(SYSDATE,d)/12);
END;
/
```

Exercise 4 - Scenario 2

```
CREATE OR REPLACE FUNCTION CalculateMonthlyInstallment(  
amt NUMBER,rate NUMBER,yrs NUMBER)  
RETURN NUMBER IS  
BEGIN  
RETURN (amt+(amt*rate*yrs/100))/(yrs*12);  
END;  
/
```

Exercise 4 - Scenario 3

```
CREATE OR REPLACE FUNCTION HasSufficientBalance(  
id NUMBER,amt NUMBER)  
RETURN BOOLEAN IS  
bal NUMBER;  
BEGIN  
SELECT Balance INTO bal FROM Accounts WHERE AccountID=id;  
RETURN bal>=amt;  
END;  
/
```

Exercise 5 - Scenario 1

```
CREATE OR REPLACE TRIGGER UpdateCustomerLastModified  
BEFORE UPDATE ON Customers  
FOR EACH ROW  
BEGIN  
:NEW.LastModified:=SYSDATE;  
END;  
/
```

Exercise 5 - Scenario 2

```
CREATE TABLE AuditLog(  
LogID NUMBER GENERATED BY DEFAULT AS IDENTITY,  
TransactionID NUMBER,LogDate DATE);  
CREATE OR REPLACE TRIGGER LogTransaction  
AFTER INSERT ON Transactions  
FOR EACH ROW  
BEGIN  
INSERT INTO AuditLog(TransactionID,LogDate)  
VALUES(:NEW.TransactionID,SYSDATE);  
END;  
/
```

Exercise 5 - Scenario 3

```
CREATE OR REPLACE TRIGGER CheckTransactionRules
BEFORE INSERT ON Transactions
FOR EACH ROW
DECLARE bal NUMBER;
BEGIN
    SELECT Balance INTO bal FROM Accounts WHERE AccountID=:NEW.AccountID;
    IF :NEW.TransactionType='Withdrawal' AND :NEW.Amount>bal THEN
        RAISE_APPLICATION_ERROR(-20002,'Insufficient Balance');
    ELSIF :NEW.TransactionType='Deposit' AND :NEW.Amount<=0 THEN
        RAISE_APPLICATION_ERROR(-20003,'Invalid Deposit');
    END IF;
END;
/
```

Exercise 6 - Scenario 1

```
DECLARE
CURSOR c IS SELECT * FROM Transactions
WHERE TRUNC(TransactionDate,'MM')=TRUNC(SYSDATE,'MM');
BEGIN
    FOR r IN c LOOP
        DBMS_OUTPUT.PUT_LINE(r.AccountID||' '||r.Amount);
    END LOOP;
END;
/
```

Exercise 6 - Scenario 2

```
DECLARE
CURSOR c IS SELECT AccountID FROM Accounts;
BEGIN
    FOR r IN c LOOP
        UPDATE Accounts SET Balance=Balance-100 WHERE AccountID=r.AccountID;
    END LOOP;
END;
/
```

Exercise 6 - Scenario 3

```
DECLARE
CURSOR c IS SELECT LoanID FROM Loans;
BEGIN
    FOR r IN c LOOP
        UPDATE Loans SET InterestRate=InterestRate+0.5
```

```

WHERE LoanID=r.LoanID;
END LOOP;
END;
/

```

Exercise 7 – Scenario 1

```

CREATE OR REPLACE PACKAGE CustomerManagement AS
  PROCEDURE AddCustomer(id NUMBER,n VARCHAR2,d DATE,b NUMBER);
  PROCEDURE UpdateCustomer(id NUMBER,n VARCHAR2);
  FUNCTION GetBalance(id NUMBER) RETURN NUMBER;
END;
/

CREATE OR REPLACE PACKAGE BODY CustomerManagement AS
  PROCEDURE AddCustomer(id NUMBER,n VARCHAR2,d DATE,b NUMBER) IS
  BEGIN INSERT INTO Customers VALUES(id,n,d,b,SYSDATE); END;
  PROCEDURE UpdateCustomer(id NUMBER,n VARCHAR2) IS
  BEGIN UPDATE Customers SET Name=n WHERE CustomerID=id; END;
  FUNCTION GetBalance(id NUMBER) RETURN NUMBER IS x NUMBER;
  BEGIN SELECT Balance INTO x FROM Customers WHERE CustomerID=id; RETURN x; END;
END;
/

```

Exercise 7 – Scenario 2

```

CREATE OR REPLACE PACKAGE EmployeeManagement AS
  PROCEDURE HireEmployee(id NUMBER,n VARCHAR2,p VARCHAR2,s NUMBER,d VARCHAR2,h
DATE);
  PROCEDURE UpdateEmployee(id NUMBER,s NUMBER);
  FUNCTION CalculateAnnualSalary(id NUMBER) RETURN NUMBER;
END;
/

CREATE OR REPLACE PACKAGE BODY EmployeeManagement AS

  PROCEDURE HireEmployee(id NUMBER,n VARCHAR2,p VARCHAR2,s NUMBER,d VARCHAR2,h
DATE) IS
  BEGIN
    INSERT INTO Employees VALUES(id,n,p,s,d,h);
  END;

  PROCEDURE UpdateEmployee(id NUMBER,s NUMBER) IS
  BEGIN
    UPDATE Employees SET Salary=s WHERE EmployeeID=id;
  END;

  FUNCTION CalculateAnnualSalary(id NUMBER) RETURN NUMBER IS a NUMBER;
  BEGIN

```

```
    SELECT Salary*12 INTO a FROM Employees WHERE EmployeeID=id;
    RETURN a;
END;

END;
/
```

Exercise 7 – Scenario 3

```
CREATE OR REPLACE PACKAGE AccountOperations AS
    PROCEDURE OpenAccount(id NUMBER,cid NUMBER,t VARCHAR2,b NUMBER);
    PROCEDURE CloseAccount(id NUMBER);
    FUNCTION GetTotalBalance(cid NUMBER) RETURN NUMBER;
END;
/

CREATE OR REPLACE PACKAGE BODY AccountOperations AS

    PROCEDURE OpenAccount(id NUMBER,cid NUMBER,t VARCHAR2,b NUMBER) IS
    BEGIN
        INSERT INTO Accounts VALUES(id,cid,t,b,SYSDATE);
    END;

    PROCEDURE CloseAccount(id NUMBER) IS
    BEGIN
        DELETE FROM Accounts WHERE AccountID=id;
    END;

    FUNCTION GetTotalBalance(cid NUMBER) RETURN NUMBER IS x NUMBER;
    BEGIN
        SELECT SUM(Balance) INTO x FROM Accounts WHERE CustomerID=cid;
        RETURN NVL(x,0);
    END;

END;
/
```